

Assembly Language Fundamentals (x86 Processors)

Prepared by: Dr. Shakil

3 - 1

Chapter Overview

- Basic Elements of Assembly Language
- Example: Adding and Subtracting Integers
- Assembling, Linking, and Running Programs
- Defining Data
- Symbolic Constants
- 64-Bit Programming

3 - 2

Basic Elements of Assembly Language

- Integer constants
- Integer expressions
- Character and string constants
- Reserved words and identifiers
- Directives and instructions
- Labels
- Mnemonics and Operands
- Comments
- Examples

3 - 3

Simple Assembly Language Program (AddTwo Program)

```
1: main PROC
2: mov eax,5 ; move 5 to the eax register
3: add eax,6 ; add 6 to the eax register
4:
5: INVOKE ExitProcess,0 ; end the program
6: main ENDP
```

Line 5 calls a window service (also known as function) named as ExitProcess that halts the program and returns control to the Operating System

3 - 4

Adding a variable

```
1: .data ; this is the data area (called data segment as well)
2: sum DWORD 0 ; create a variable named sum (size: 32 bits)
3:
4: .code ; this is the code area (called code segment as well)
5: main PROC
6: mov eax,5 ; move 5 to the eax register
7: add eax,6 ; add 6 to the eax register
8: mov sum,eax
9:
10: INVOKE ExitProcess,0 ; end the program
11: main ENDP
```

3 - 5

Integer Constants (Integer Literals)

- leading + or – sign (optional)
- One or more digits
- binary, decimal, hexadecimal, or octal digits
- Common radix characters: (optional) It indicates number's base
 - h – hexadecimal
 - q/o- Octal
 - d – decimal (alternate: t)
 - b – binary (alternate: y)
 - r – encoded real

3 - 6

Examples: 26 is a valid integer literal. By default decimal. If we write 1101 then it will be considered as decimal as well.

Examples

26, 26d, 1101b, 42q, 42o, 1Ah, 0A3h

Note: A hexadecimal literal beginning with a letter must have a leading zero to prevent the assembler from interpreting it as an identifier.

3 - 7

Constant Integer Expressions

A constant integer expression is a mathematical expression involving integer literals and arithmetic operators. Each expression must evaluate to an integer, which can be stored in 32 bits (0 through FFFFFFFh)

- Operators and precedence levels:

Operator	Name	Precedence Level
()	parentheses	1
+ -	unary plus, minus	2
* /	multiply, divide	3
MOD	modulo	3
+ -	add, subtract	4

- Examples:

Expression	Value
16 / 5	3
-(3 + 4) * (6 - 1)	-35
-3 + 4 * 6 - 1	20
25 mod 3	1

Suggestion: Use parentheses in expressions to clarify the order of operations so you don't have to remember precedence rules

3 - 8

Real Number Literals

- Will be covered later on

3 - 9

Character and String Constants (Literals)

- Enclose character in single or double quotes
 - 'A', "x"
 - ASCII character = 1 byte (A=65 or 41 hex)
- Enclose strings (including spaces) in single or double quotes
 - 'ABC', "ABC"
 - 'xyz'
 - '4096'
 - "Good night, Gracie"
 - Each character occupies a single byte

3 - 10

Embedded quotes

- Embedded quotes are permitted when used in the manner shown by the following examples:
 - "This isn't a test"
 - 'Say "Good night," Gracie'
- Just as character constants are stored as integers, we can say that string literals are stored in memory as sequences of integer byte values. So, for example, the string literal "A BCD" contains the four bytes 41h, 42h, 43h, and 44h.

3 - 11

Reserved Words and Identifiers

- Reserved words cannot be used as identifiers (by default not case sensitive)
 - Instruction mnemonics (such as MOV, ADD etc),
 - Register names
 - Directives (which tell the assembler how to assemble programs)
 - type attributes (which provide size and usage information for variables and operands. Examples are BYTE and WORD)
 - operators (used in constant expressions)
 - predefined symbols (such as @data, which return constant integer values at assembly time)
 - See MASM reference in Appendix A
- Identifiers
 - An identifier is a programmer-chosen name. It might identify a variable, a constant, a procedure, or a code label
 - 1-247 characters, including digits
 - not case sensitive
 - first character must be a letter, _, @, ?, or \$
 - Can not be same as assembler reserved word

3 - 12

Directives

- Commands that are recognized and acted upon by the assembler
 - Not part of the Intel instruction set
 - Used to declare code, data areas, select memory model, declare procedures, etc.
 - not case sensitive
- Different assemblers have different directives
 - NASM not the same as MASM, for example

3 - 13

- The DWORD directive tells the assembler to reserve space in the program for a doubleword variable.
- The MOV instruction, on the other hand, executes at runtime, copying the contents of myVar to the EAX register:
 - myVar DWORD 26
 - mov eax,myVar
- Although all assemblers for Intel processors share the same instruction set, they usually have different sets of directives

3 - 14

Segments

- One important function of assembler directives is to define program sections, or segments. Segments are sections of a program that have different purposes. For example, one segment can be used to define variables, and is identified by the .DATA directive:

```
.data
```

- The .CODE directive identifies the area of a program containing executable instructions:

```
.code
```

- The .STACK directive identifies the area of a program holding the runtime stack, setting its size:

```
.stack 100h
```

- Appendix A contains a useful reference for directives and operators.

3 - 15

Instructions

- Assembled into machine code by assembler
- Executed at runtime by the CPU
- We use the Intel IA-32 instruction set
- Syntax: [label:] mnemonic [operands] [:comment]
- An instruction contains:
 - Label (optional)
 - Mnemonic (required)
 - Operand (depends on the instruction)
 - Comment (optional)

3 - 16

Labels

- Act as place markers
- marks the address (offset) of code and data
- Follow identifier rules
- Data label
 - must be unique
 - The assembler assigns a numeric address to each label
 - Example: count DWORD 100
 - example: **myArray** (not followed by colon)
 - array DWORD 1024, 2048
DWORD 4096, 8192

3 - 17

- Code label
 - A label in the code area of a program (where instructions are located) must end with a colon (:) character.
 - Code labels are used as targets of jumping and looping instructions. For example, the following JMP (jump) instruction transfers control to the location marked by the label named target, creating a loop:


```
target:
    mov ax,bx

    ..

    jmp target
```
 - A code label can share the same line with an instruction, or it can be on a line by itself:


```
L1: mov ax,bx
L2:
```

3 - 18

Mnemonics and Operands

• Instruction Mnemonics

- memory aid
- examples: MOV, ADD, SUB, MUL, INC, DEC, JMP, CALL

• Operands

- Assembly language instructions can have between zero and three operands, each of which can be a register, memory operand, integer expression, or input-output port
- constant
- constant expression
- register
- memory (data label)

Constants and constant expressions are often called **immediate values**

3 - 19

• Instruction Examples (0-3 operands)

- stc ; set Carry flag
- inc eax ; add 1 to EAX
- mov count, ebx ; move EBX to count
- imul eax,ebx,5

- First one is destination operand and remaining are source operands

3 - 20

Comments (1 of 2)

• Comments are good!

- explain the program's purpose
- when it was written, and by whom
- revision information
- tricky coding techniques
- application-specific explanations

• Single-line comments

- begin with semicolon (;)

3 - 21

Comments (2 of 2)

• Multi-line comments (Block Comments)

- begin with COMMENT directive and a programmer-chosen character

- End with the same programmer-chosen character

COMMENT !

This line is a comment.

This line is also a comment.

!

- We can also use any other symbol, as long as it does not appear within the comment lines:

COMMENT &

This line is a comment.

This line is also a comment.

&

3 - 22

Instruction Format Examples

• No operands

- stc ; set Carry flag

• One operand

- inc eax ; register
- inc myByte ; memory

• Two operands

- add ebx,ecx ; register, register
- sub myByte,25 ; memory, constant
- add eax,36 * 25 ; register, constant-expression

3 - 23

What's Next (1 of 5)

- Basic Elements of Assembly Language
- **Example: Adding and Subtracting Integers**
- Assembling, Linking, and Running Programs
- Defining Data
- Symbolic Constants
- 64-Bit Programming

3 - 24

Example: Adding and Subtracting Integers

```
; AddTwo.asm – adds two 32-bit integers
.386 ; identifies that it is a 32 bit program
.model flat,stdcall ; flat is memory model. Stdcall is calling convention
.stack 4096 ; sets 4096 bytes for storage of runtime stack
ExitProcess PROTO, dwExitCode:DWORD
.code
main PROC
    mov eax,5 ; move 5 to the EAX register
    add eax,6 ; add 6 to the EAX register
    INVOKE ExitProcess,0 ; 0 means successful return
main ENDP ; ENDP is the directive
END main
```

3 - 25

Example Output

Showing registers and flags in the debugger:

```
EAX=00030000 EBX=7FFDF000 ECX=00000101 EDX=FFFFFFFF
ESI=00000000 EDI=00000000 EBP=0012FFF0 ESP=0012FFC4
EIP=00401024 EFL=00000206 CF=0 SF=0 ZF=0 OF=0
```

3 - 26

Suggested Coding Standards (1 of 2)

- Some approaches to capitalization
 - capitalize nothing
 - capitalize everything
 - capitalize all reserved words, including instruction mnemonics and register names
 - capitalize only directives and operators
- Other suggestions
 - descriptive identifier names
 - spaces surrounding arithmetic operators
 - blank lines between procedures

3 - 27

Suggested Coding Standards (2 of 2)

- Indentation and spacing
 - code and data labels – no indentation
 - executable instructions – indent 4-5 spaces
 - comments: right side of page, aligned vertically
 - 1-3 spaces between instruction and its operands
 - ex: movax,bx
 - 1-2 blank lines between procedures

3 - 28

Required Coding Standards

- (to be filled in by the professor)

3 - 29

Program Template

```
; Program Template (Template.asm)
; Program Description:
; Author:
; Creation Date:
; Revisions:
; Date: Modified by:
.386
.model flat,stdcall
.stack 4096
ExitProcess PROTO, dwExitCode:DWORD
.data
; declare variables here
.code
main PROC
; write your code here
INVOKE ExitProcess,0
main ENDP
; (insert additional procedures here)
END main
```

3 - 30

What's Next (2 of 5)

- Basic Elements of Assembly Language
- Example: Adding and Subtracting Integers
- **Assembling, Linking, and Running Programs**
- Defining Data
- Symbolic Constants
- 64-Bit Programming

3 - 31

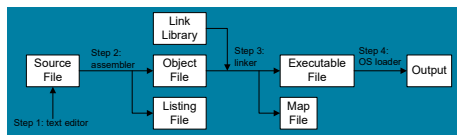
Assembling, Linking, and Running Programs

- Assemble-Link-Execute Cycle
- Listing File
- Map File

3 - 32

Assemble-Link Execute Cycle

- The following diagram describes the steps from creating a source program through executing the compiled program.
- If the source code is modified, Steps 2 through 4 must be repeated.



3 - 33

Listing File

- Use it to see how your program is compiled
- Contains
 - source code
 - addresses
 - object code (machine language)
 - segment names
 - symbols (variables, procedures, and constants)

3 - 34

What's Next (3 of 5)

- Basic Elements of Assembly Language
- Example: Adding and Subtracting Integers
- Assembling, Linking, and Running Programs
- **Defining Data**
- Symbolic Constants
- 64-Bit Programming

3 - 35

Defining Data (1 of 2)

- Intrinsic Data Types
- Data Definition Statement
- Defining BYTE and SBYTE Data
- Defining WORD and SWORD Data
- Defining DWORD and SDWORD Data
- Defining QWORD Data
- Defining TBYTE Data

3 - 36

Defining Data (2 of 2)

- Defining Real Number Data
- Little Endian Order
- Adding Variables to the AddSub Program
- Declaring Uninitialized Data

3 - 37

Intrinsic Data Types (1 of 2)

- BYTE, SBYTE (B stands for Byte, S stands for signed)
 - 8-bit unsigned integer; 8-bit signed integer
- WORD, SWORD
 - 16-bit unsigned & signed integer
- DWORD, SDWORD (D stands for Double)
 - 32-bit unsigned & signed integer
- QWORD (Q stands for quad)
 - 64-bit integer
- TBYTE (T stands for Ten-byte)
 - 80-bit integer

3 - 38

Intrinsic Data Types (2 of 2)

- REAL4
 - 4-byte (32-bit) IEEE short real
- REAL8
 - 8-byte (64-bit) IEEE long real
- REAL10
 - 10-byte (80-bit) IEEE extended real

3 - 39

Data Definition Statement

- A data definition statement sets aside storage in memory for a variable.
- May optionally assign a name (label) to the data
- Syntax:
`[name] directive initializer [,initializer] . . .`
 value1 BYTE 10
- All initializers become binary data in memory

3 - 40

Defining BYTE and SBYTE Data

Each of the following defines a single byte of storage:

value1 BYTE 'A' ; character constant
 value2 BYTE 0 ; smallest unsigned byte
 value3 BYTE 255 ; largest unsigned byte
 value4 SBYTE -128; smallest signed byte
 value5 SBYTE +127 ; largest signed byte
 value6 BYTE ? ; uninitialized byte (runtime assignment)

- MASM does not prevent you from initializing a BYTE with a negative value, but it's considered poor style.
- If you declare a SBYTE variable, the Microsoft debugger will automatically display its value in decimal with a leading sign.

3 - 41

Defining Byte Arrays

Examples that use multiple initializers:

list1 BYTE 10,20,30,40
 list2 BYTE 10,20,30,40
 BYTE 50,60,70,80
 BYTE 81,82,83,84
 list3 BYTE ?,32,41h,00100010b
 list4 BYTE 0Ah,20h,'A',22h

Memory layout of a byte sequence.

Offset	Value
0000:	10
0001:	20
0002:	30
0003:	40

3 - 42

Defining Strings (1 of 3)

- A string is implemented as an array of characters
 - For convenience, it is usually enclosed in quotation marks
 - It often will be *null-terminated*

- Examples:

```
str1 BYTE "Enter your name",0
str2 BYTE 'Error: halting program',0
str3 BYTE 'A','E','I','O','U'
greeting BYTE "Welcome to the Encryption Demo
program "
BYTE "created by Kip Irvine.",0
```

3 - 43

Defining Strings (2 of 3)

- To continue a single string across multiple lines, end each line with a comma:

```
menu BYTE "Checking Account",0dh,0ah,0dh,0ah,
    "1. Create a new account",0dh,0ah,
    "2. Open an existing account",0dh,0ah,
    "3. Credit the account",0dh,0ah,
    "4. Debit the account",0dh,0ah,
    "5. Exit",0ah,0ah,
    "Choice> ",0
```

3 - 44

Defining Strings (3 of 3)

- End-of-line character sequence:
 - 0Dh = carriage return
 - 0Ah = line feed

```
str1 BYTE "Enter your name: ",0Dh,0Ah
BYTE "Enter your address: ",0
newLine BYTE 0Dh,0Ah,0
```

Idea: Define all strings used by your program in the same area of the data segment.

3 - 45

Using the DUP Operator

- Use DUP to allocate (create space for) an array or string. Syntax: *counter* DUP (*argument*)
- Counter* and *argument* must be constants or constant expressions

```
var1 BYTE 20 DUP(0) ; 20 bytes, all equal to zero
var2 BYTE 20 DUP(?) ; 20 bytes, uninitialized
var3 BYTE 4 DUP("STACK"); 20 bytes:
"STACKSTACKSTACKSTACK"
var4 BYTE 10,3 DUP(0),20; 5 bytes
```

3 - 46

Defining WORD and SWORD Data

- Define storage for 16-bit integers
 - or double characters
 - single value or multiple values

```
Word1 WORD 65535 ; largest unsigned value
Word2 SWORD -32768 ; smallest signed value
Word3 WORD ? ; uninitialized, unsigned
Word4 WORD "AB" ; double characters
myList WORD 1,2,3,4,5 ; array of words
Array WORD5 DUP(?) ; uninitialized array
```

3 - 47

Defining DWORD and SDWORD Data

Storage definitions for signed and unsigned 32-bit integers:

```
val1 DWORD 12345678h ; unsigned
val2 SDWORD -2147483648 ; signed
val3 DWORD20 DUP(?) ; unsigned array
val4 SDWORD -3,-2,-1,0,1 ; signed array
```

3 - 48

Defining QWORD, TBYTE, Real Data

Storage definitions for quadwords, tenbyte values, and real numbers:

```
quad1 QWORD 1234567812345678h
val1 TBYTE 1000000000123456789Ah
rVal1 REAL4 -2.1
rVal2 REAL8 3.2E-260
rVal3 REAL10 4.6E+4096
ShortArray REAL4 20 DUP(0.0)
```

3 - 49

Little Endian Order

- All data types larger than a byte store their individual bytes in reverse order. The least significant byte occurs at the first (lowest) memory address.

- Example:

```
val1 DWORD 12345678h
```

0000:	78
0001:	56
0002:	34
0003:	12

3 - 50

Adding Variables to AddSub

```
TITLE Add and Subtract, Version 2(AddSub2.asm)
; This program adds and subtracts 32-bit unsigned
; integers and stores the sum in a variable.
INCLUDE Irvine32.inc
.data
val1 DWORD 10000h
val2 DWORD 40000h
val3 DWORD 20000h
finalVal DWORD ?
.code
main PROC
mov eax, val1      ; start with 10000h
add eax, val2      ; add 40000h
sub eax, val3      ; subtract 20000h
mov finalVal, eax  ; store the result (30000h)
call DumpRegs     ; display the registers
exit
main ENDP
END main
```

3 - 51

Declaring Uninitialized Data

- Use the `.data?` directive to declare an uninitialized data segment:

```
.data?
```

- Within the segment, declare variables with `"?"` initializers:

```
smallArray DWORD 10 DUP(?)
```

Advantage: the program's EXE file size is reduced.

3 - 52

What's Next (4 of 5)

- Basic Elements of Assembly Language
- Example: Adding and Subtracting Integers
- Assembling, Linking, and Running Programs
- Defining Data
- Symbolic Constants**
- 64-Bit Programming

3 - 53

Symbolic Constants

- A symbolic constant (or symbol definition) is created by associating an identifier (a symbol) with an integer expression or some text. Symbols do not reserve storage. They are used only by the assembler when scanning a program, and they cannot change at runtime. The following table summarizes their differences:

	Symbol	Variable
Uses storage?	No	Yes
Value changes at runtime?	No	Yes

- Equal-Sign Directive
- Calculating the Sizes of Arrays and Strings
- EQU Directive
- TEXTEQU Directive

3 - 54

Equal-Sign Directive

- *name* = *expression*
- *expression* is a 32-bit integer (expression or constant)
- may be redefined
- *name* is called a **symbolic constant**
- good programming style to use symbols

```
COUNT = 500
```

```
.
```

```
.
```

```
mov ax,COUNT
```

3 - 55

Calculating the Size of a Byte Array

- current location counter: \$
 - subtract address of list
 - difference is the number of bytes

```
list BYTE 10,20,30,40
```

```
ListSize = ($ - list)
```

In the above example, ListSize is calculated by subtracting the offset of list from the current location counter (\$):

- ListSize must follow immediately after list

3 - 56

Calculating the Size of a Word Array

When calculating the number of elements in an array containing values other than bytes, you should always divide the total array size (in bytes) by the size of the individual array elements. The following code, for example, divides the address range by 2 because each word in the array occupies 2 bytes (16 bits)

Example

```
list WORD 1000h,2000h,3000h,4000h
```

```
ListSize = ($ - list) / 2
```

3 - 57

Calculating the Size of a Doubleword Array

Divide total number of bytes by 4 (the size of a doubleword)

```
list DWORD 1,2,3,4
```

```
ListSize = ($ - list) / 4
```

3 - 58

EQU Directive

- Define a symbol as either an integer or text expression.

- Cannot be redefined

```
PI EQU <3.1416>
```

```
pressKey EQU <"Press any key to continue...",0>
```

```
.data
```

```
prompt BYTE pressKey
```

3 - 59

Example

- Example Suppose we would like to define a symbol that counts the number of cells in a 10-by-10 integer matrix. We will define symbols two different ways, first as an integer expression and second as a text expression. The two symbols are then used in data definitions:

- matrix1 EQU 10 * 10

- matrix2 EQU <10 * 10>

- .data

- M1 WORD matrix1

- M2 WORD matrix2

- The assembler produces different data definitions for M1 and M2. The integer expression in matrix1 is evaluated and assigned to M1. On the other hand, the text in matrix2 is copied directly into the data definition for M2:

- M1 WORD 100

- M2 WORD 10 * 10

3 - 60

TEXTEQU Directive

- Define a symbol as either an integer or text expression.
 - Called a **text macro**
 - Can be redefined
- ```

continueMsg TEXTEQU <"Do you wish to continue (Y/N)?">
rowSize = 5
.data
prompt1 BYTE continueMsg
count TEXTEQU %(rowSize * 2) ; evaluates the expression
setupAL TEXTEQU <mov al,count>
.code
setupAL ; generates: "mov al,10"

```

3 - 61

### What's Next (5 of 5)

- Basic Elements of Assembly Language
- Example: Adding and Subtracting Integers
- Assembling, Linking, and Running Programs
- Defining Data
- Symbolic Constants
- **64-Bit Programming**

3 - 62

### 64-Bit Programming

- MASM supports 64-bit programming, although the following directives are not permitted:
  - INVOKE, ADDR, .model, .386, .stack
  - (Other non-permitted directives will be introduced in later chapters)

3 - 63

### 64-Bit Version of AddTwoSum

```

1: ; AddTwoSum_64.asm - Chapter 3 example.
3: ExitProcess PROTO
5: .data
6: sum DWORD 0
8: .code
9: main PROC
10: mov eax,5
11: add eax,6
12: mov sum,eax
13:
14: move cx,0
15: call ExitProcess
16: main ENDP
17: END

```

3 - 64

### Things to Notice About the Previous Slide

- The following lines are not needed:
 

```

.model flat,stdcall
.stack 4096

```
- INVOKE is not supported.
- CALL instruction cannot receive arguments
- Use 64-bit registers when possible

3 - 65

### Summary

- Integer expression, character constant
- directive – interpreted by the assembler
- instruction – executes at runtime
- code, data, and stack segments
- source, listing, object, map, executable files
- Data definition directives:
  - BYTE, SBYTE, WORD, SWORD, DWORD, SDWORD, QWORD, TBYTE, REAL4, REAL8, and REAL10
  - DUP operator, location counter (\$)
- Symbolic constant
  - EQU and TEXTEQU

3 - 66